

Welcome: One Real Problem, Three Courses, One Team

Friday, September 25, 2026 · Phase 1 – Analysis · 1 h lecture · 2 h workshop · 2 h at home

Asen Grozdanov (Cyber) · Cyber.SoHo Educational Hub · From the Industry to the Classroom

1

Microeconomics

what is an hour worth?

2

Statistics

what does the time log show?

3

Python / OOP

how do we automate and
measure it?



Today's hour — four parts, four activities

1

0:00–0:11

Welcome and introductions

Then: Sticky-note wall (0:11–0:15)

2

0:15–0:26

What is an integrative project?

Then: Venn diagram + fun quiz (0:26–0:30)

3

0:30–0:41

Course map, grading and the Brick system

Then: Flash-cards + zen break (0:41–0:45)

4

0:45–0:56

Meet the client and the toolbox

Then: Open the CSV: day-1 question (0:56–1:00)

Then the 2-hour workshop (Brick 1 “Kick-off Pack”, checkpoint at 2:45) and 2 hours at home. Every Friday has this same 1 : 2 : 2 shape.



By the end of this class you can...

1

Explain in one sentence what JARVIS must do for Boréal Studio by January.

JARVIS is the name of our assistant — a name, not an abbreviation.

2

Name the six key dates and the 30 / 30 / 40 grading split without looking.

Nov 13 · Nov 20 · Dec 4, 11, 18 · Jan 15 · Jan 22 · 30 % Bricks, 30 % Mid-Term, 40 % Final.

3

Run a first Python script that reads the routines dataset from a team repository — and log your first routine.

That is Brick 1, in the workshop today.



Before we start — the materials and the rules

Cyber.SoHo materials

This deck, the lecture module, the demos, the figures, the tables and the datasets are educational material of the Cyber.SoHo Educational Hub.

By using them you accept the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (v2.0).

Read this week: section 2 (acceptance), section 4 (intellectual property), section 18 (academic integrity).

Use for your own study and coursework: yes. Redistribution, sale or presenting them as your own: no.

The University's rules

The official syllabus and the University's policies apply in full: code of conduct, attendance, academic honesty, resit.

They are linked from the syllabus on the **LMS** (Learning Management System).

Cite every source — code, documentation pages and **AI** (Artificial Intelligence) assistance — in **APA** (American Psychological Association) 7 style.

A time log is personal data: it never leaves the private team repository.

- [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy \(v2.0\) — open the document](#)



1

Welcome and introductions

In this part

- Who we are — from the industry to the classroom
- Introductions: name, program, one routine you would love to never do again
- Expectations, both ways
- The one rule of the course: no new theory — integration



Welcome — and now you: name, program, one routine

From the industry to the classroom

About twenty-five years in the **IT** (Information Technology) industry before teaching.

In industry nobody hands you a problem labelled “statistics” or “economics” or “programming”.

You receive a business problem and decide which knowledge answers which part of it.

Your turn — in one breath

Your **name**, your **program**, and **one routine you would love to never do again**.

I write the routines on the board: by the end of the round we have twenty or thirty of them.

What is a routine?

- A **routine** is a task that repeats on a regular schedule — every day, every week — and follows more or less the same steps each time.
- Checking a mailbox every morning: a routine.
- Writing the same status report every Friday: a routine.
- Registering for a course once a year: not a routine — an event.
- The whole project is about routines: the ones that can be automated, and the ones that cannot.



Expectations — both ways

What I expect from you, every week

You **attend** the Friday session: one hour of lecture, two hours of team work — the team work cannot happen without you.

You **submit the Brick** — the one deliverable of the week — by Thursday 23:59, on the **LMS** and on GitHub.

You **commit code** every week. Commits from every member count; the history shows who typed.

You **ask questions**. If I do not know the answer, it goes on the Parking Lot and is answered next Friday.

What you can expect from me

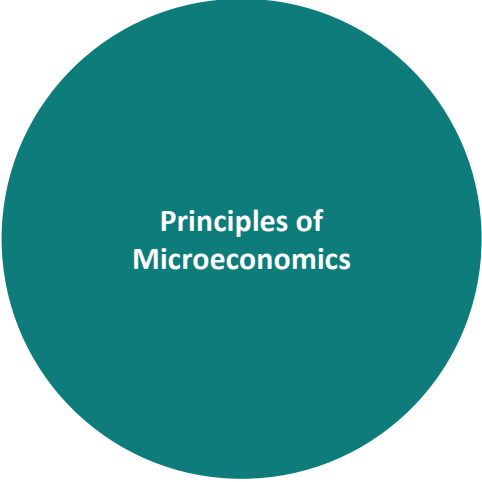
The **same rhythm** every Friday, so you always know what comes next.

A **grade for every Brick within a week** — complete, partial or missing, plus one comment.

One specific, true, positive sentence to every team, every week.



The one rule: no new theory — integration



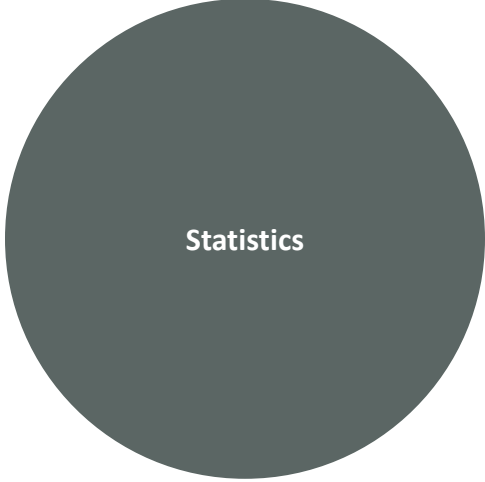
Principles of
Microeconomics

opportunity cost, marginal benefit vs cost, break-even



Introduction to OOP
in Python

classes, objects, the csv / statistics / sqlite3
modules



Statistics

mean, median, standard deviation, correlation

- **This course adds no new theory.** Everything you need was taught in three courses you have already taken.
- What is new is the **integration**: using the three kinds of knowledge together, on one real problem, until you have a working tool and a recommendation a business owner can act on.
- Keep three words: **one problem, three courses, one team.**



Activity — the sticky-note wall

4
min

- Take **one sticky note**.
- Write **one thing you still remember** from one of the three courses — Microeconomics, Python, or Statistics.
- Stick it on the board **under that course**.
- Three notes are read aloud. The wall stays up all semester: it is the first version of the knowledge map we use in Part 2.

“opportunity cost = the value of the next-best option”

Microeconomics

“a class is a template for objects”

Python

“the median is the middle value”

Statistics



Watch at home — project-based learning in four minutes

Five Keys to Rigorous Project-Based Learning (Edutopia)

A short explanation of project-based learning - the format of this course. Map the five keys to CS 251: real-world connection = Boréal Studio; core to learning = the three courses; structured collaboration = teams of 2-3; student-driven = your own time log and design choices; multifaceted assessment = Bricks, Mid-Term and Product Defense.

- https://www.youtube.com/watch?v=hnzCGNnU_WM · about 4 min · YouTube

Before you click

Links were checked on September 24, 2026.

Paste any address into VirusTotal or Google Safe Browsing first.

Third-party sites and their advertising are outside Cyber.SoHo's control (Notice, sections 14 and 15, Annex C).

- [VirusTotal — URL check](#)



Real-life example — Alex's first day in the time log

- **Task.** Alex, a first-year **CS** (Computer Science) student, records every study and work routine of Friday, September 25 as one row: *date, routine, minutes, interruptions*.
- Then: total the minutes per routine, find the routine that took the most time, write the rows as a **CSV** (Comma-Separated Values) file.
- **Why here.** In the introductions you named one routine; the time log turns routines into data.
- From today every student is a **pilot user**: a personal **14-day time log**, two minutes a day.
- **Brick 1** (due Oct 1): `data/timelog_<initials>.csv`, at least five rows per member. **Brick 4** (Oct 22): your log analysed with the same statistics as the studio's data.

StartNext stepRestartSpeedNormalCopy this page's codeFinished - press Restart to run again

Minutes per routine (grows as rows are added)

Alex's time log - Friday 2026-09-25 - minutes per routine

Download chart (SVG)

Download chart (PNG)

date	routine	minutes	interruptions
2026-09-25	email_check	25	2
2026-09-25	lecture_notes	50	1
2026-09-25	coding_practice	40	3
2026-09-25	group_chat	15	4
2026-09-25	email_check	10	0
2026-09-25	commute_reading	30	0

Execution log - every step is recorded here

Step 10: Count interruptions per routine

Interruptions will be compared with minutes in Brick 4 (correlation). Interruptions: email_check=2, lecture_notes=1, coding_practice=3, group_chat=4, commute_reading=0 - group_chat is the most interrupted per minute.

Step 11: Write the CSV text

CSV (Comma-Separated Values): one line per row, commas between values, a header line first. CSV written: 1 header line + 6 data lines. Download it with the button below.

Step 12: Where this goes

Brick 1: the log started (>= 5 rows). Brick 4: your 14-day log analysed (mean, SD, correlation). Privacy: no names, no message contents. Done. Keep logging 2 minutes a day for 14 days.

CSV text of the log (this is what data/timelog_<initials>.csv contains)

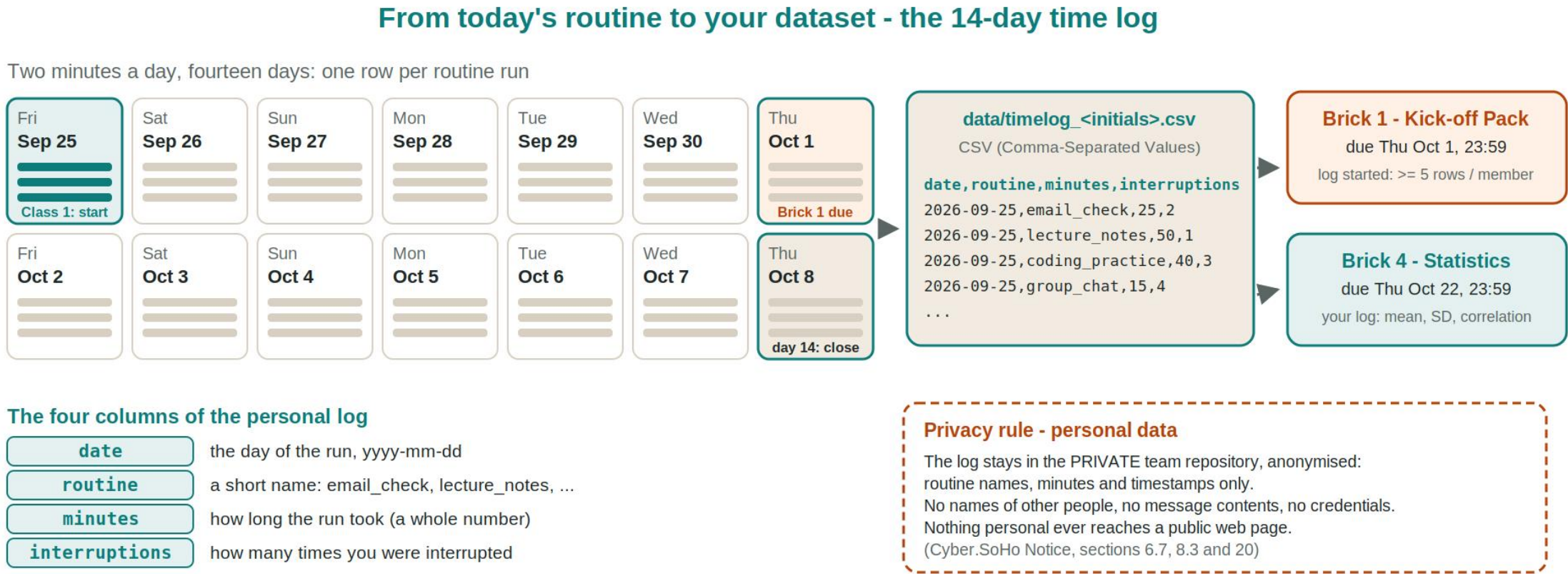
```
date,routine,minutes,interruptions
2026-09-25,email_check,25,2
2026-09-25,lecture_notes,50,1
2026-09-25,coding_practice,40,3
2026-09-25,group_chat,15,4
2026-09-25,email_check,10,0
2026-09-25,commute_reading,30,0
```

Download CSV

Demo 1 — Time Log Builder (HTML, runs offline): Start / Pause, Next step, Restart, Copy, execution log, downloads. Result: `lecture_notes` = 50 min is the longest routine of Alex's day.



Figure 1 — from today's routine to your dataset



Grouping the rows by routine name and adding the minutes is the first computation of Brick 4; the same grouping answers 'which routine took the most minutes?' for the studio's data in Part 4.

Figure 1 - CS 251, Class 1, Part 1 - Cyber.SoHo Educational Hub, 2026



The privacy rule — a time log is personal data

Say it once today, repeat it every week

The log stays in the **private** team repository, **anonymised**.

Routine names, minutes and timestamps only.

No names of other people, no message contents, no credentials.

Nothing personal ever reaches a public web page — the published dashboard shows aggregated numbers only.

Where it is written

Cyber.SoHo Notice — section 6.7 (data and privacy in laboratories), section 8.3 (privacy and data protection), section 20 (personal information).

Never commit an **API** (Application Programming Interface) key: it lives in `.env` locally and in a repository secret on GitHub.

A secret that was ever committed is rotated the same day.

- [Cyber.SoHo Notice — sections 6.7, 8.3 and 20](#)



Remember — Part 1

1

What is the one rule of this course about theory?

No new theory. The course integrates three courses already taken: Principles of Microeconomics, Introduction to OOP in Python, and Statistics.

2

What is a routine?

A task that repeats on a regular schedule and follows more or less the same steps each time - for example, triaging e-mail every morning.

3

What does every student start today and keep for 14 days?

A personal time log: date, routine, minutes, interruptions - about two minutes of logging per day.

4

Where does a personal time log live?

In the private team repository, anonymised: routine names, minutes and timestamps only; no names, no message contents; never on a public page.

5

What is a Brick?

The one deliverable of the week, due Thursday 23:59 on the LMS and GitHub; twelve Bricks make the 30 % continuous assessment and assemble into the project.



2

What is an integrative project?

In this part

- The definition, word by word
- The consulting habit: problem, evidence, recommendation
- Research problem → sub-problems → research questions
- Why three courses together — and what JARVIS must do by January



The definition, word by word (1/2)

“a sustained period of independent study and research, combined with interaction with other students”

Two things at once: you work independently on research, and you work with a team.

“starts with a real-world business problem, provided by the instructor or by an industrial partner”

This year the problem is provided by me; the business is a small Montréal consultancy you meet in Part 4.



The definition, word by word (2/2)

“a research-based solution is sought, integrating the knowledge acquired in previous courses, aiming to reproduce real-world consulting activities”

Three requirements: based on research, not opinion; integrate what you already know; reproduce what a consultant does for a client.

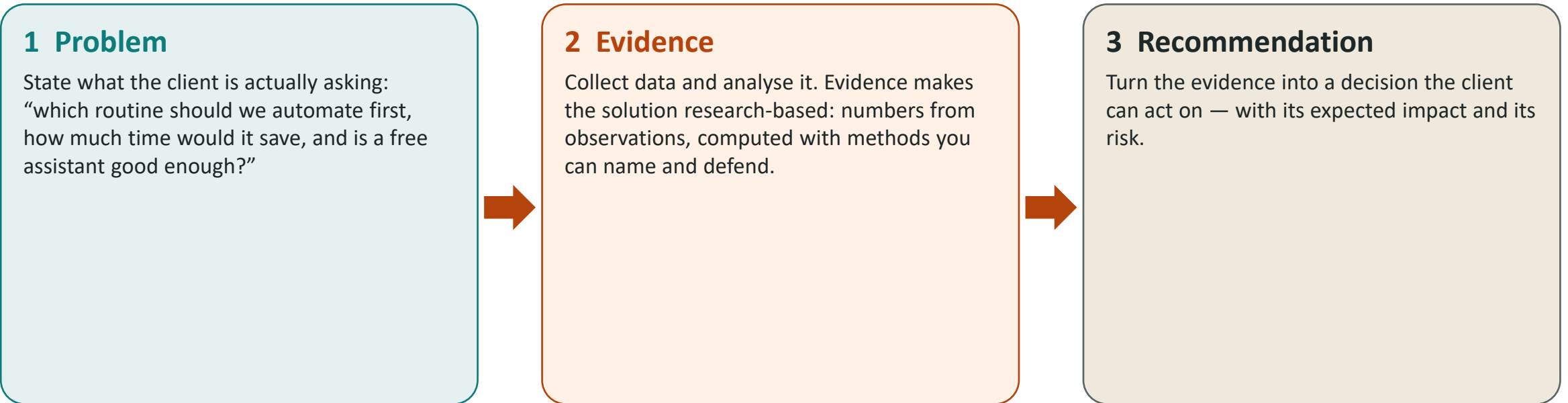
“the course does not include additional theoretical content”

The example given for this year: a Python application that automates the routine work of a small company, records real usage data, and applies statistics and microeconomics to that data to make business decisions.



The consulting habit: problem, evidence, recommendation

A **consulting engagement** is a contract in which a client asks an outside expert to analyse a problem and recommend a decision. The client does not want a lecture; the client wants to know what to do.



We keep that order in every Brick and in the Product Defense of January 22. In Class 2 you write your own problem statement; today, just recognise the shape.



The research side: from the problem to the research questions

- Our textbook, Leedy and Ormrod, *Practical Research: Planning and Design* (12th ed.), describes the research side of the same order.
- A **research problem** is split into **sub-problems**; each sub-problem becomes a **research question (RQ)**; data is collected to answer each question.
- In Class 2 you write your own problem statement and three research questions (Brick 2, due October 8).
- The template below is refined by every team in Brick 2.

RQ1 — Statistics

Which routines cost the most time, and which are the most variable?

RQ2 — Microeconomics

What is an hour of staff time worth, what does automation cost, and where is the break-even between the free tier and a paid assistant?

RQ3 — Integration

Which routines should JARVIS automate first, how much time does it save, and what is the risk?



Why three courses together

What is an hour of staff time worth, and what does automation cost?

Principles of Microeconomics

- **opportunity cost** — the value of the next-best use of a resource
- **marginal benefit** vs **marginal cost**
- fixed and variable costs
- **break-even** — where benefit equals cost

What does the time log actually show?

Statistics

- **mean** — sum divided by count
- **median** — middle value of the sorted data
- **standard deviation (SD)** — how spread out the values are
- **correlation**, before / after comparison

How do we automate a routine and measure the saving?

OOP (Object-Oriented Programming) in Python

- **classes** — data bundled with the operations on it
- **objects** — instances of a class
- the standard library: csv, statistics, sqlite3

None of the three answers the owner's question on its own: economics without data is a guess; statistics without a tool is a spreadsheet nobody maintains; a program without economics automates the wrong routine.



Figure 2 — Microeconomics ∩ Python ∩ Statistics = CS 251

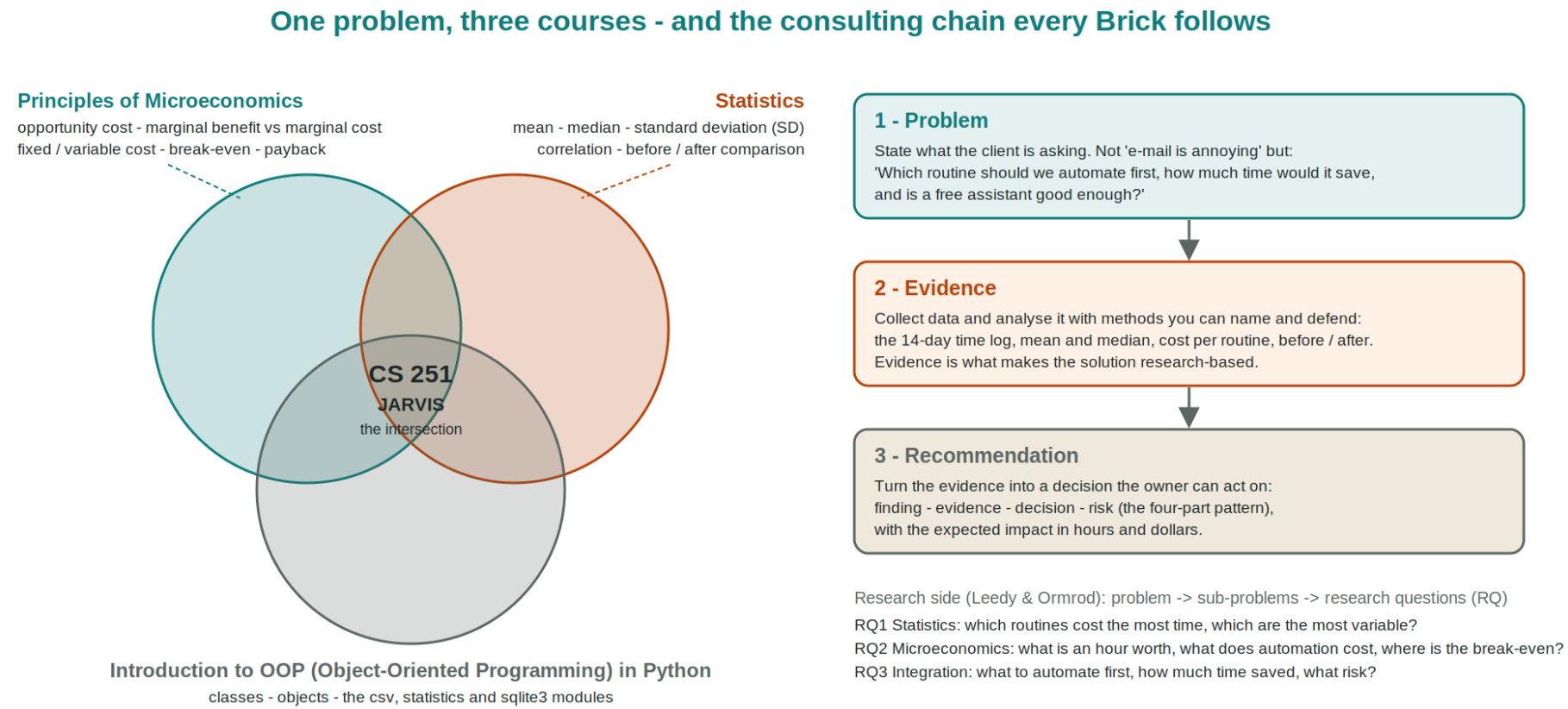


Figure 2 - CS 251, Class 1, Part 2 - Cyber.SoHo Educational Hub, 2026

JARVIS — the name of the assistant you will build (a name, not an abbreviation) — lives in the intersection: it logs and automates routine work, measures the time it saves with statistics, prices that time with microeconomics, and recommends what to automate first and whether the free tier is enough.



Activity — the three-circle diagram and a fun quiz

4
min

Hands up — which course gives us...

1

... opportunity cost?

2

... the median?

3

... a class with an `__init__` method?

Harder, for the whole room: which circle does “the monthly cost of e-mail triage, computed by a Python method from the median minutes per day” belong to?



Watch at home — how consultants solve a problem

How do consulting firms work - hypothesis-based problem solving explained (Firm Learning)

A former consultant explains the order problem -> hypothesis -> evidence -> recommendation. Watch it at home; in class we use only the order itself, which is the backbone of every Brick and of the Product Defense.

- <https://www.youtube.com/watch?v=TBvJzXxRuxs> · about 10 min · YouTube

Keep from the video

The order: problem → hypothesis → evidence → recommendation.

It is the backbone of every Brick and of the Product Defense.

Check links before opening them (VirusTotal); third-party sites are outside Cyber.SoHo's control.

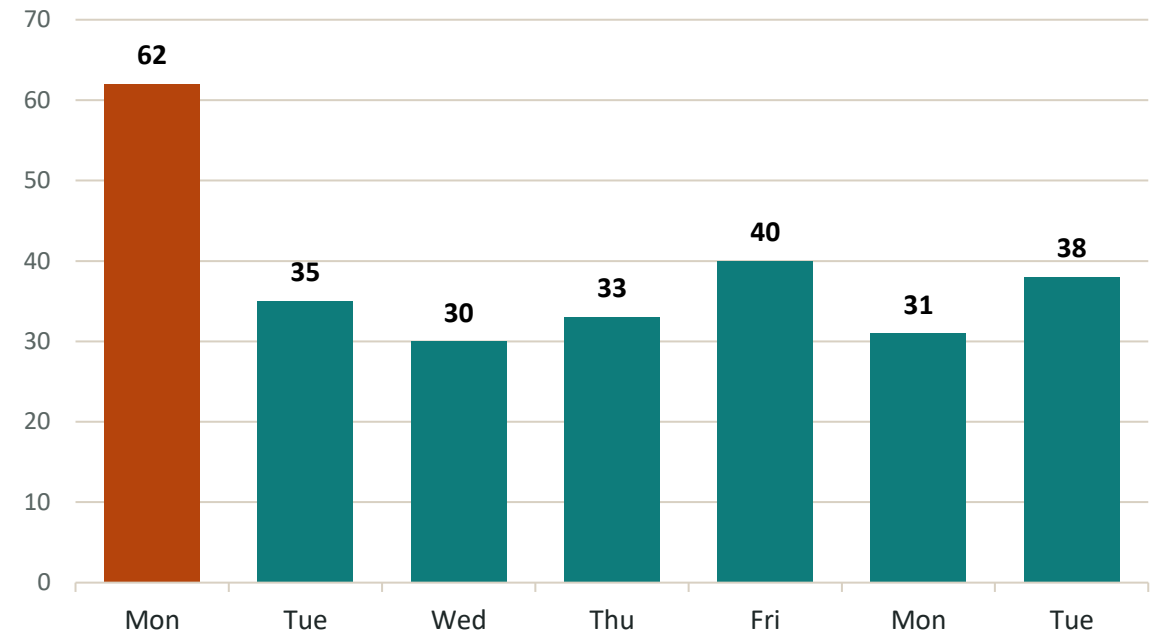
- [VirusTotal — URL check](#)



Real-life example — e-mail triage at Boréal Studio

- **Problem.** The owner: “E-mail triage eats our mornings. Should we automate it first, and is a free assistant good enough?”
- **Evidence.** Seven days of triage minutes for one staff member: 62, 35, 30, 33, 40, 31, 38.
- **Statistics.** Mean = $269 / 7 = 38.4$ min; median = **35** min. Monday (62) is a backlog: the median is the honest typical day.
- **Microeconomics.** $35 / 60 \times 28 \$/h \times 20 \text{ days} \times 5 \text{ staff} \approx 1\,633 \$$ per month.
- **Python / OOP.** The same computation becomes one method: `EconEngine.routine_cost(...)` — Bricks 3 and 6.

E-mail triage — minutes per day (one staff member, 7 days)



Read the chart

The rust bar (Monday backlog) pulls the mean up to 38.4; the median stays at 35 — the value a consultant uses when one day is extreme.



Demo 2 — the chain in code, and the recommendation

Start

Next step

Restart

Speed

Normal

Copy this page's code

Finished - press Restart to run again

The chain and the three courses (lights up as the steps run)

Consulting chain: problem -> evidence -> recommendation

Problem

→

Evidence

→

Recommendation

Micro ∩ Stats ∩ Python = CS 251

mean 36.4
median 35

E-mail triage, minutes per day (7 days, one staff member)

rust dot = Monday backlog (62 min), an outlier

Download figure (SVG)

Download figure (PNG)

Recommendation (four-part pattern)

Finding: e-mail triage costs Boreal Studio about 1 633 \$ per month (median 35 min/day, 5 staff, 20 days).

Evidence: 7-day time log of one staff member; median used because Monday is an outlier.

Decision: e-mail triage is the first candidate to automate; confirm with the pilot rows (tool = jarvis from day 10) in Brick 3.

Risk: 14 days and 5 people are a small sample; the free API quota may not cover every request; personal data must stay anonymised.

Execution log - every step is recorded here

Step 8: Python/OOP - the computation becomes a method
EconEngine.routine_cost(minutes, hourly_value, runs) will compute this for every routine in Brick 3 and Brick 6.
All three circles are lit: the answer needs the three courses together.

Step 9: Recommendation - Finding
Finding = the number.
Finding written.

Step 10: Recommendation - Evidence
Evidence = where the number comes from.
Evidence written.

Step 11: Recommendation - Decision
Decision = what the owner does.
Decision written.

Step 12: Recommendation - Risk
Risk = what could go wrong.
Risk written. The chain is complete: this is the pattern of every Brick and of the final Product Defense.

Finding

e-mail triage costs about 1 633 \$ per month (median 35 min/day, 5 staff, 20 days).

Evidence

7-day time log of one staff member; median used because Monday is an outlier.

Decision

e-mail triage is the first candidate to automate; confirm with the pilot rows (tool = jarvis from day 10) in Brick 3.

Risk

14 days and 5 people are a small sample; the free API quota may not cover every request; personal data must stay anonymised.

Demo 2 — Consulting Chain Simulator: 12 steps; the Venn circles light up as each course is used.



Remember — Part 2

1

Define an integrative project in one sentence.

Independent study and research with a team, starting from a real business problem and leading to a research-based solution that integrates previous courses and reproduces consulting work - with no new theory.

2

What are the three steps of the consulting habit?

Problem -> evidence -> recommendation: state what the client asks, collect and analyse data, then recommend a decision with its impact and risk.

3

Which course supplies opportunity cost, marginal benefit and break-even?

Principles of Microeconomics.

4

Which course supplies mean, median, standard deviation and correlation?

Statistics.

5

What are the three research questions of JARVIS (template)?

RQ1: which routines cost the most time and are the most variable? (Statistics) - RQ2: what is an hour worth, what does automation cost, where is the break-even? (Microeconomics) - RQ3: which routines to automate first, how much time saved, what risk? (Integration).



3

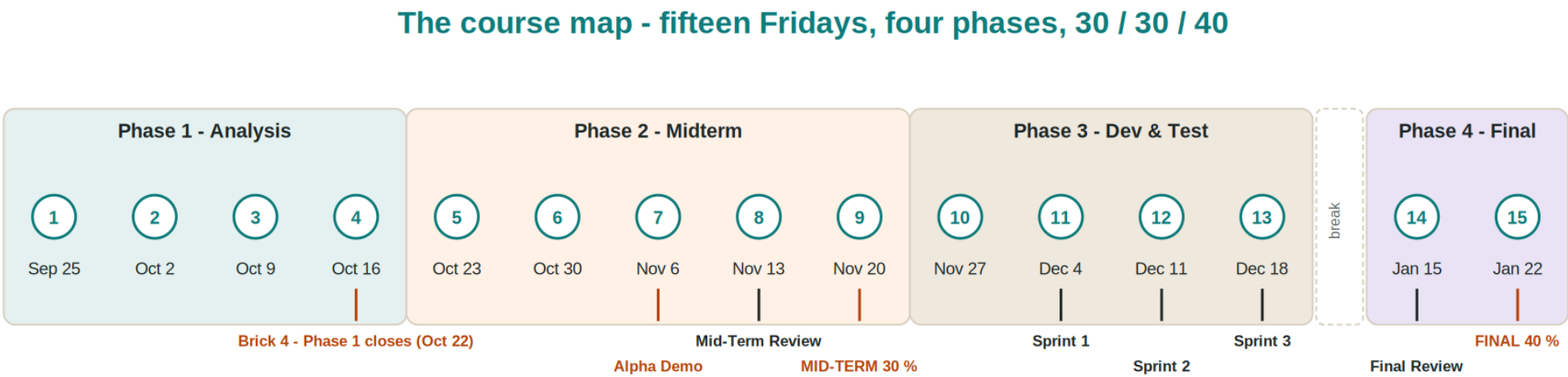
Course map, grading and the Brick system

In this part

- Fifteen Fridays in four phases — and the six dates to memorise
- The rhythm 1 : 2 : 2
- Grading 30 / 30 / 40 and the letter scale
- The Brick system — why every Brick is a page of your Mid-Term



Fifteen Fridays, four phases



Every Friday, the same rhythm - 1 : 2 : 2

1 h lecture 3-4 short parts, an activity after each	2 h in-class workshop team work on the Brick, checkpoint at the end	2 h at home finish the Brick, short reading, Weekly Progress Report
--	--	--

Grading - 30 / 30 / 40 (letter grades: 93 A - 90 A- - 87 B+ - 83 B - 80 B- - 77 C+ - 73 C - 70 C- - 60 D - below 60 F)

Continuous 30 % - 12 Bricks x 2.5 %	Mid-Term 30 % - Fri Nov 20, 2-hour exam	Final 40 % - Fri Jan 22, Product Defense
-------------------------------------	---	--

Bricks: Complete 2.5 - Partial 1.25 - Missing 0 (must still be submitted). Due Thursday 23:59 on the LMS + GitHub. Bricks 1-7 complete by Nov 12: the Mid-Term is written on them.
Final submission Thu Jan 21, 23:59 (report PDF, slides PDF, codebase tag v1.0-final). Winter break Dec 21 - Jan 8.

Figure 3 - CS 251, Class 1, Part 3 - Cyber.SoHo Educational Hub, 2026



Six dates to memorise — and two that matter soon

Nov 13

Mid-Term Review

Nov 20

MID-TERM 30 %

Dec 4 · 11 · 18

Project Sprints 1, 2, 3

Jan 15

Final Review

Jan 22

FINAL 40 %

Two more that matter soon

Thursday, October 22

Brick 4 due — Phase 1 (Project Proposal and Requirement Analysis) closes.

Friday, November 6

Alpha Demo — the intermediate presentation (5 minutes per team).

All classes on Fridays. Bricks are due Thursday 23:59 before the next class. Final submission: Thursday, January 21, 23:59 (tag v1.0-final).



The rhythm: 1 : 2 : 2 — every Friday has the same shape

1 h

Lecture

- three or four short parts of 11–15 minutes
- after each part, a 3–5-minute activity: a visualization, a short practical, flash-cards, a quiz or a zen break

2 h

In-class workshop

- your team works on the week's Brick
- the last 15 minutes are the **checkpoint**: every team shows the day's result on screen, 30–60 seconds

2 h

At home

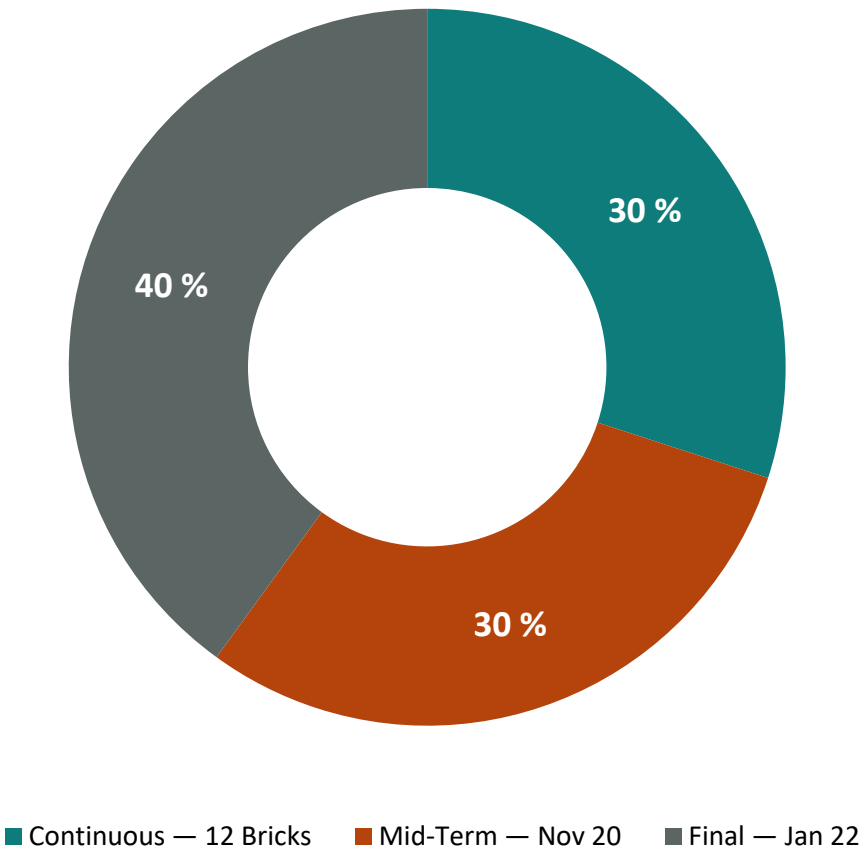
- finish the Brick
- a short extract of Leedy and Ormrod
- the **Weekly Progress Report** (ten lines, ten minutes)

Three credit hours = 135 to 180 study hours over the semester; the rhythm above is how those hours are distributed. Predictability is deliberate.



Grading: 30 / 30 / 40

Weight of the three components



Continuous Course Assessments — 30 %

Twelve Bricks at 2.5 % each: Complete 2.5 · Partial 1.25 · Missing 0, plus one comment. Progress reports, commits and peer review count too.

Midterm Assessment — 30 % — Friday, November 20

Two hours, individual, computational and application. Part A: costs, break-even, payback, mean, median, SD, correlation. Part B: complete or fix a Python method, read a design diagram, write requirements and research questions.

Final Assessment — 40 % — Friday, January 22

The Product Defense: live demonstration of the working assistant, public presentation with guests; technical report and codebase submitted January 21 at 23:59.



Letter grades — the syllabus scale

Percentage	Letter
93 – 100 %	A
90 – 92 %	A-
87 – 89 %	B+
83 – 86 %	B
80 – 82 %	B-

Percentage	Letter
77 – 79 %	C+
73 – 76 %	C
70 – 72 %	C-
60 – 69 %	D
59 % and below	F

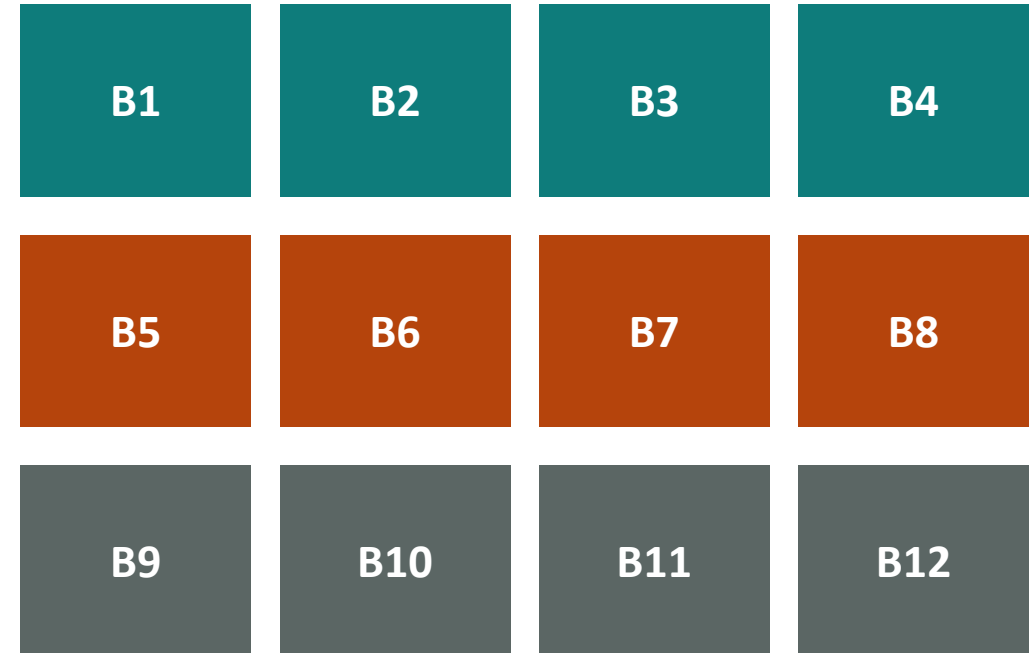
Assessment levels used in feedback (five rubric dimensions)

Exemplary 93–100 % · Proficient 83–92 % · Developing 70–82 % — Methodology, Tech Implementation, Analytical Rigor, Lab Proficiency, Communication. Used for the Alpha Demo feedback, the peer review, the dress rehearsal and the Final.



The Brick system — twelve weeks, twelve Bricks, one project

- A **Brick** is the one deliverable of the week: a document, a script, a diagram, a set of tests.
- Twelve Bricks, twelve weeks. They are not separate homework: they **assemble into JARVIS**.
- Nothing is thrown away — not even the Mid-Term: its corrected Part A becomes the first draft of the report's Results section.
- **The one fact that decides the Mid-Term:** the exam is written on the JARVIS basis — the same routines data, the same formulas you coded in Bricks 3 and 4, the same design you drew in Brick 5.
- Every Brick submitted on time is a page of your own preparation; every gap is a gap in the exam. **Bricks 1 to 7 complete by Thursday, November 12.**



Bricks 1–4: Phase 1 (proposal and requirements) · Bricks 5–8: the Alpha and the Mid-Term basis · Bricks 9–12: tests, going live, results, report draft.



Brick rules — in five lines

1

Due Thursday 23:59

before the next class: files on the LMS (Learning Management System), code on GitHub, tagged where the class page says so.

2

One team submission + one Weekly Progress Report per student

ten lines, ten minutes: what I did, what the team delivered, what I will do next, blockers, hours spent, one thing I learned.

3

Commits from every member count

Git activity is part of the continuous assessment.

4

A missing Brick must still be submitted

the late policy is on the LMS; the Mid-Term and the report are built on every Brick.

5

Cite your sources in APA 7

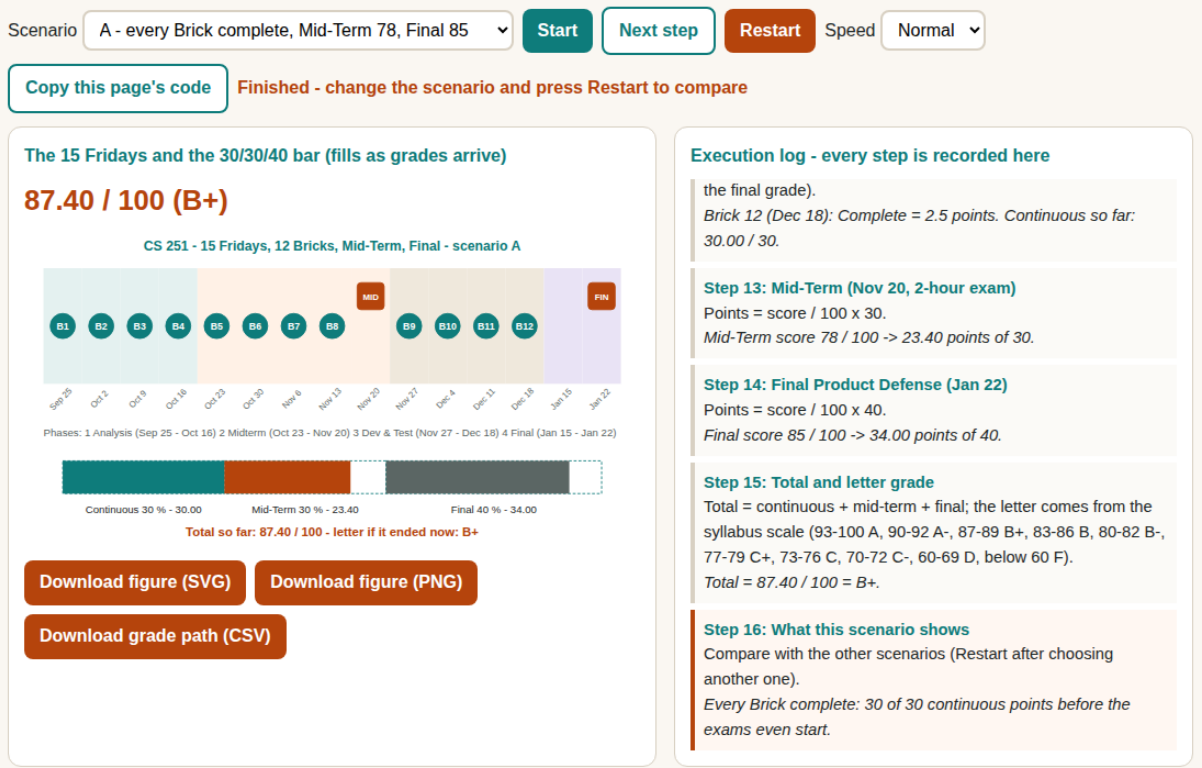
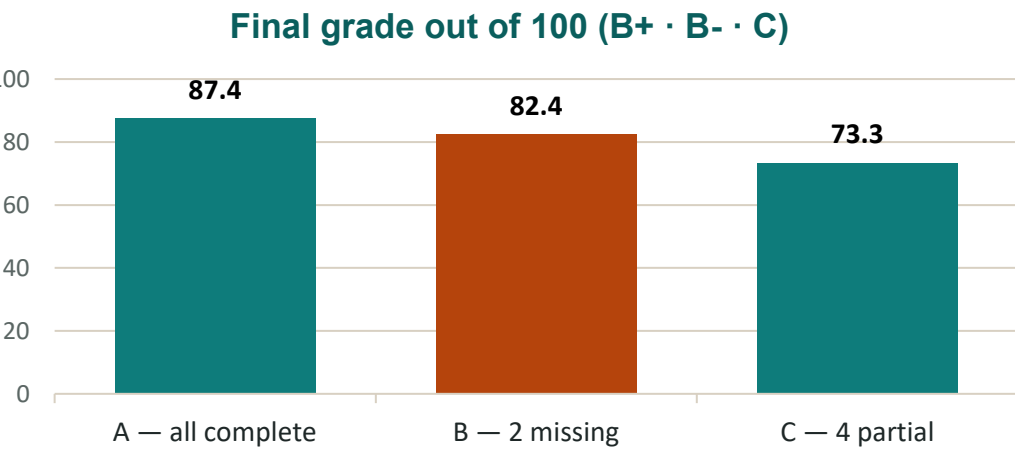
code, documentation pages and AI (Artificial Intelligence) assistance included. Reused code without a citation is an academic-honesty case (University policy; Cyber.SoHo Notice, section 18).

- [Cyber.SoHo Notice — section 18, academic integrity and submitted work](#)



Real-life example — Maya's grade path

- **Task.** Compute Maya's final grade week by week: 12 Bricks (2.5 / 1.25 / 0), then Mid-Term $\times 0.30$, then Final $\times 0.40$; read the letter on the syllabus scale.
- **Scenario A** — every Brick complete, Mid-Term 78, Final 85.
- **Scenario B** — Bricks 5 and 9 missing, same exams.
- **Scenario C** — four partial Bricks, Mid-Term 65, Final 72.



Demo 3 — Grade Path Simulator. Same exam marks, two missing Bricks: **five points less, B+ becomes B-** — before counting the Mid-Term questions written on those Bricks. Turning a written rule into a computation is the habit of Brick 3 (formulas → EconEngine) and Brick 6 (the Reporter).



Activity — flash-cards and a zen break

4
min

Flash-cards, in pairs (2 min)

One student holds the cards, the other answers; swap after one round.

November 13?

November 20?

December 4, 11, 18?

January 15?

January 22?

30 / 30 / 40?

Zen break (2 min)

Stand up, stretch.

Breathe on a **4-4-4** rhythm: inhale for four seconds, hold for four, exhale for four.

Ten cycles make two minutes — the breathing timer in the lecture module runs them for you.



Watch at home — short delivery cycles (Scrum)

Introduction to Scrum - 7 Minutes (Uzility)

Explains how software teams deliver in short, fixed cycles that each produce a usable increment. Our Bricks are weekly increments, our checkpoint is a short review, and December 4, 11 and 18 are literally called Project Sprints.

- <https://www.youtube.com/watch?v=9TycLR0TqFA> · 7 min · YouTube

Keep from the video

Our Bricks are weekly increments; our checkpoint is a short review.

December 4, 11 and 18 are literally called Project Sprints.

Check links before opening them (VirusTotal).

- [VirusTotal — URL check](#)



Remember — Part 3

1

What is the grading split of the course?

Continuous 30 % (12 Bricks × 2.5 %), Mid-Term 30 % (November 20), Final Product Defense 40 % (January 22).

2

What does the 1:2:2 rhythm mean?

1 hour of lecture in short parts with activities, 2 hours of in-class workshop with a checkpoint, 2 hours at home (Brick, reading, Weekly Progress Report).

3

When and what is the Mid-Term?

Friday, November 20, 2026: a 2-hour individual computational and application exam written on the JARVIS basis (Bricks 1-7).

4

When and what is the Final?

Friday, January 22, 2027: the Product Defense - live demo, public presentation, technical report and codebase (submitted January 21, 23:59).

5

What happens to a missing Brick?

It scores 0 but must still be submitted: the Mid-Term and the report are built on it. Late policy as announced on the LMS.



4

Meet the client and the toolbox

In this part

- Boréal Studio — six routines, five people, one question
- The dataset: boreal_routines_2026.csv and its nine columns
- The three given numbers
- The toolbox (Python only) and the JARVIS loop with its offline rule



The client — Boréal Studio, Montréal

- A **five-person consultancy** — a teaching case: the company is fictional, the data is synthetic, so no real person's working hours are exposed.
- It loses hours every week to routines nobody was hired for.
- Six routines, five people, **fourteen working days** of timed data.
- You are part of the case too: every student is a pilot user with a 14-day time log.

email_triage sorting the mailbox, deciding what needs an answer

daily_report writing the end-of-day report

scheduling finding meeting slots

invoice_check checking invoices

standup_notes notes for the daily team meeting

social_post posting on social media

“Which of these routines should we automate first, how much time would it really save, and is a free assistant good enough — or should we pay for one?” — the owner. She wants a tool that works, not a lecture.



The dataset — one row per routine run

Column	Meaning	Python type
date	the day of the run, written yyyy-mm-dd	datetime.date
staff_id	an anonymous code for the person (S1 ... S5)	str
routine	the name of the routine	str
category	communication, reporting, planning, finance, marketing	str
start_hour	the hour the run started, 0–23	int
minutes	how long the run took	int
interruptions	how many times the person was interrupted	int
tool	manual, template or jarvis	str
done	completed: 1 yes, 0 no	bool (0/1)

CSV (Comma-Separated Values)

Plain text, one line per record, values separated by commas, a header line naming the columns.

About **420 rows** = 14 working days × 5 staff × about 6 routines.

Two deliberate events

A **process change**: a report template introduced on day 8.

A **pilot automation**: assistant-triaged e-mail from day 10.

They exist so that a before/after comparison and a correlation can be computed honestly.



The three given numbers

28 \$ / h

hourly value of staff time

a **loaded** cost: salary plus the employer's overhead per hour worked

**20 \$ / user /
month**

price of a paid assistant tier

a fixed cost per period

free API quota

requests allowed on the free tier

API = Application Programming Interface. Free tiers change: the number and the date it was read are posted on the LMS

These three numbers feed every cost, break-even and payback computation of Brick 3 and of the EconEngine; you will record the quota and its date in your economics notes.



The toolbox — Python only

1

Python 3.12+

and its standard library: csv, sqlite3, statistics, dataclasses

3

Virtual environment

`python -m venv .venv` — a private copy of Python per project; pip (Package Installer for Python) installs into it

5

pytest and matplotlib

the test framework (from Class 10) and the charting library for the report

2

VS Code

Visual Studio Code with the Python extension — an IDE (Integrated Development Environment)

4

Git and GitHub

every version of your files; the private team repository, GitHub Actions (scripts on a schedule) and GitHub Pages (a public page)

6

The LMS and the textbook

submissions; Leedy and Ormrod, Practical Research: Planning and Design, 12th edition

No Java is needed anywhere in this course.



The JARVIS loop — and the offline fallback rule



The log is JARVIS's own dataset: in December it feeds the “time saved” numbers (Brick 11).

OfflineProvider — mandatory, the default

Rules and **regular expressions** (patterns that match text), deterministic, **no key, no network**.

Tests, the Mid-Term and the demos must never depend on the internet.

ApiProvider — an upgrade, never a dependency

Connected in Class 10: a free **LLM** (Large Language Model) tier through an **API** (Application Programming Interface).

Drops into the same loop; on any error it falls back to the offline provider.



Figure 4 — dataset, three numbers and the loop

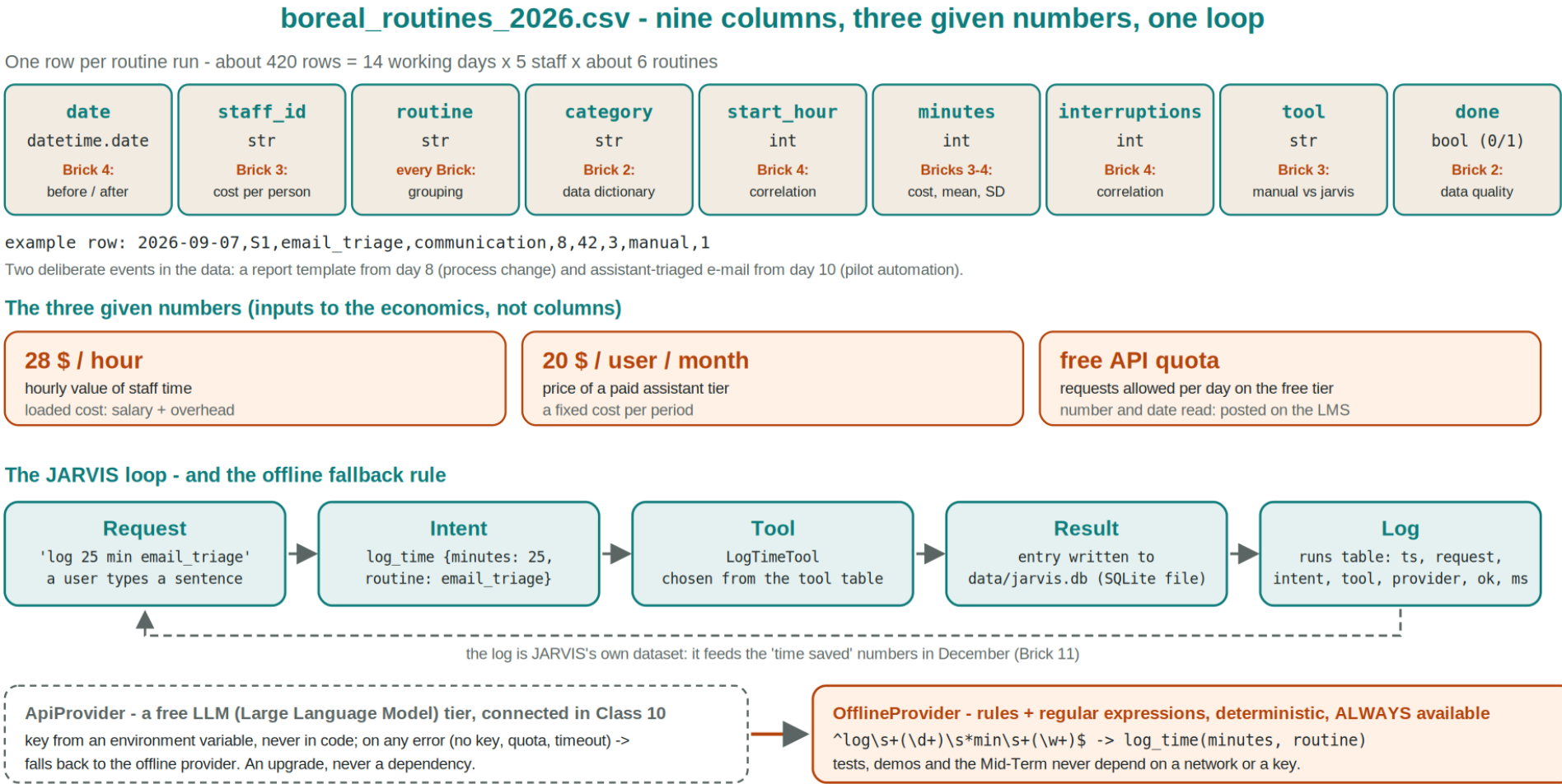
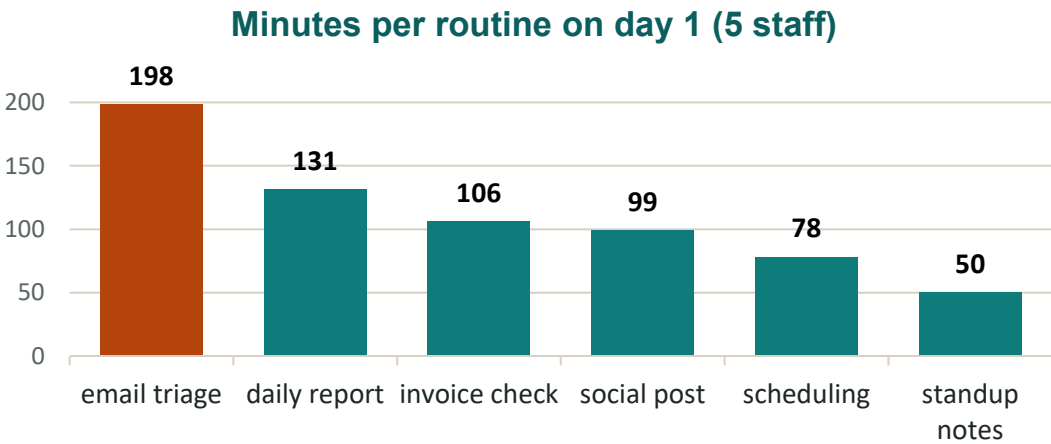


Figure 4 - CS 251, Class 1, Part 4 - Cyber.SoHo Educational Hub, 2026 - Boreal Studio is a fictional teaching case; the data is synthetic.



Example — JARVIS hears “log 25 min email_triage”

- **Task.** Read the day-1 preview (30 rows: 5 staff × 6 routines), print the first three rows like `hello_jarvis.py`, group the minutes per routine, then run one request through the five stages of the loop — offline — and record the run.
- **Bricks.** 1 (`hello_jarvis.py` reads the CSV), 5 (`loader.py` parses rows safely), 7 (`OfflineProvider`, `Tool`, `Agent` — the Alpha Demo).



StartNext stepRestartSpeedNormalRequestlog 25 min email_triageCopy this page's code

Finished - change the request and press Restart to try another

The loop (active stage in rust) and minutes per routine on day 1

The JARVIS loop: request -> intent -> tool -> result -> log

Request

Intent

Tool

Result

Log

the log feeds the next request (and the statistics of Brick 11)

Minutes per routine on day 1 (2026-09-07, 5 staff) - the Part 4 activity question

198

131

78

106

50

99

email_triage

daily_report

scheduling

invoice_check

standup_notes

social_post

Download figure (SVG)

Download figure (PNG)

Download day-1 preview (CSV)

First rows of the file (what `hello_jarvis.py` prints in Brick 1)

date,staff_id,routine,category,start_hour,minutes,interruptions,tool,done
2026-09-07,S1,email_triage,communication,8,42,3>manual,1
2026-09-07,S1,daily_report,reporting,16,25,1>manual,1
2026-09-07,S1,daily_report,reporting,16,25,1>manual,1
2026-09-07,S1,scheduling,planning,18,15,2>manual,1

JARVIS reply and run record

JARVIS: Logged 25 min of email_triage for 2026-09-25.

run record: {
 "ts": "2026-09-24T21:53:43.762Z",
 "request": "log 25 min email_triage",
 "intent": "log_time",
 "tool": "LogTimeTool",
 "provider": "Offline",
 "ok": 1,
 "ms": 3
}

Execution log - every step is recorded here

Step 6: tool (lookup in the tool table)
The Agent looks up the intent name in its tool table: (log_time, add_task, daily_report).
Tool selected: LogTimeTool - a subclass of the Tool base class (polymorphism preview, Brick 5).

Step 9: Result (the tool runs)
LogTimeTool appends a new entry to the store; in the real JARVIS the Store writes it into data/jarvis.db.
Entry appended. The store now holds 31 records.

Step 10: Log (the run record)
Every request is recorded in the runs table: timestamp, request, intent, tool, provider, ok, milliseconds. This table is JARVIS's own dataset for Brick 11.
Run recorded with provider = offline.

Step 11: The loop closes
Offline fallback rule: JARVIS must work with no key and no network, so demos, tests and the exam never depend on the internet. The API provider (Class 10) is a drop-in upgrade.
Ready for the next request. Bricks: 1 (hello_jarvis.py reads the CSV), 5 (loader.py parses rows safely), 7 (OfflineProvider + Agent).

Demo 4 — JARVIS Loop Simulator. The regular expression `^log\s+(\d+)\s*min\s+(\w+)\s*$` captures the minutes and the routine; a sentence that does not match gets “I did not understand” instead of a crash. Answer to the activity: **email_triage, 198 minutes**; longest single run S3, 48 minutes.



Activity — open the CSV: the day-1 question

4
min

- Open `boreal_routines_2026.csv` from the **LMS** in any text editor — or from a terminal with one line of Python:

```
python -c "import csv; r=list(csv.DictReader(open('boreal_routines_2026.csv'))); print(r[:3])"
```

- Look at the `date`, `routine` and `minutes` columns.
- **Question: which routine took the most minutes on day 1?**
- First hands on the data. The answer is checked at the checkpoint; Demo 4 shows the same computation on the preview rows.



Watch at home — Git, virtual environments and assistants

Git Explained in 100 Seconds (Fireship)

2 min · YouTube

Repository, staging, commit, branch - the four words you need for Brick 1's 'one commit from every member'.

- [Open the video](#)

Python Virtual Environments - Full Tutorial for Beginners (Tech With Tim)

9 min · YouTube

Why a virtual environment exists and how to create and activate one on Windows, macOS and Linux - the first thing you do in the workshop.

- [Open the video](#)

What are AI Agents? (IBM Technology)

12 min · YouTube

The idea of an assistant that turns a request into a tool call is the JARVIS loop; the video explains the general concept, our project builds a small, offline-first version of it.

- [Open the video](#)

Links checked on September 24, 2026. Check any address with VirusTotal before opening it; third-party sites are outside Cyber.SoHo's control (Notice, sections 14 and 15).



Remember — Part 4

1

Name the nine columns of `boreal_routines_2026.csv`.

date, staff_id, routine, category, start_hour, minutes, interruptions, tool, done.

2

What are the three given numbers?

Hourly value of staff time 28 \$/h (loaded cost); paid assistant tier 20 \$/user/month; the free API quota, with the date it was read.

3

What are the five stages of the JARVIS loop?

Request -> intent -> tool -> result -> log.

4

What is the offline fallback rule?

JARVIS must run with no API key and no network: the offline provider uses rules and regular expressions; the API provider is an upgrade, never a dependency.

5

What is a virtual environment and how is it created?

A private copy of Python for one project so that packages never conflict; created with `python -m venv .venv` and then activated.



The workshop (1:00–3:00) — Brick 1 “Kick-off Pack”

1:00

Teams and roles

teams of two or three; a team name; a first rotating role: driver (types), navigator (reads and checks), scribe (writes the charter).

1:15

Environment check

python --version shows 3.12+; VS Code with the Python extension; python -m venv .venv; hello_jarvis.py prints the team name and the first three rows of the dataset with the csv module.

1:45

Git and GitHub

the private team repository from the starter layout (jarvis/, data/, docs/, tests/, analysis/, .gitignore); a README.md; one commit from every member; add the instructor as collaborator.

2:15

Charter and time log

docs/charter.md, one page: team and roles, the problem in your own words, three questions for the owner, what “success” looks like on January 22; data/timelog_<initials>.csv started with today's routines.

2:45

Checkpoint

thirty seconds per team: show the README and the running script. Home-work briefing.



Home work (2 h) and Brick 1 — due Thursday, October 1, 23:59

At home this week

Finish Brick 1: charter v1 in the repository; the script runs on every member's machine (one screenshot each).

The data diary: log every study and work routine for 14 days — two minutes a day.

Read Leedy and Ormrod, Chapter 1, *The Nature and Tools of Research* (about 40 minutes); note three sentences that define a “research problem”.

Weekly Progress Report #1 (template on the LMS) — ten minutes.

Brick 1 deliverables (LMS + GitHub)

Team repository with README, .gitignore and one commit per member.

hello_jarvis.py running on the dataset inside a virtual environment.

docs/charter.md (one page).

Personal time log started — at least five rows per member.

Weekly Progress Report #1.

Feeds into: the JARVIS repository and README; the report's title page, team and first problem statement; your own 14-day dataset (Brick 4).

Next Friday, Class 2: The Problem Before the Code — requirements and the research process (Brick 2 due October 8).



Links used in this lecture (1/3)

- [Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy \(v2.0\)](#) — The terms that govern every Cyber.SoHo material, including this page, its demos, figures and datasets. Sections most relevant to this course: 2, 4, 5, 6.6, 6.7, 7.9, 8.3, 9, 10-16, 17, 18, 20 and Annex C.
- [Python Software Foundation - python.org](#) — The official home of Python; download Python 3.12 or newer from here only.
- [The Python 3 documentation - Tutorial and Library Reference](#) — The recommended free reference of the course: csv, sqlite3, statistics, dataclasses, venv.
- [Visual Studio Code](#) — The editor used in the workshops, with the Python extension.
- [Git](#) — The version control system; official installers and the free book Pro Git.
- [GitHub](#) — Hosts the private team repository, GitHub Actions and GitHub Pages.
- [GitHub Docs](#) — Git basics, Actions and Pages - the second recommended free resource of the syllabus.
- [GitHub Student Developer Pack](#) — Free GitHub Pro for students (needed for Pages on a private repository).
- [pytest documentation](#) — The test framework used from Class 10.

Before you click: paste the address into VirusTotal or Google Safe Browsing (Cyber.SoHo Notice, section 14 and Annex C).



Links used in this lecture (2/3)

- [Matplotlib](#) — The charting library for the report's figures (Sprint 2).
- [SQLite](#) — The database engine behind data/jarvis.db, used through Python's sqlite3 module.
- [APA Style \(American Psychological Association\)](#) — The citation style required for every source, including code, documentation pages and AI assistance.
- [Pearson - Leedy & Ormrod, Practical Research: Planning and Design \(12th ed.\)](#) — The publisher of the required textbook; borrow it from the library or buy it here.
- [Groq - free API tier](#) — One of the free LLM API tiers considered for the API provider in Class 10 (quotas change; read them on the day).
- [Google AI for Developers - Gemini API](#) — A second free-tier option for the API provider.
- [GitHub Models](#) — A third free-tier option, inside GitHub.
- [draw.io](#) — A free diagram editor for the UML class diagram of Brick 5.
- [PlantUML](#) — Text-based UML diagrams, an alternative for Brick 5.

Before you click: paste the address into VirusTotal or Google Safe Browsing (Cyber.SoHo Notice, section 14 and Annex C).



Links used in this lecture (3/3)

- [VirusTotal - URL check](#) — Check any link before you open it (Cyber.SoHo Notice, section 14 and Annex C).
- [Google Safe Browsing - site status](#) — A second reputation check for domains (Annex C).
- [Five Keys to Rigorous Project-Based Learning \(Edutopia, YouTube\)](#) — Part 1 video: the format of this course explained in about four minutes.
- [How do consulting firms work - hypothesis-based problem solving \(Firm Learning, YouTube\)](#) — Part 2 video: problem -> hypothesis -> evidence -> recommendation.
- [Introduction to Scrum - 7 Minutes \(Uzility, YouTube\)](#) — Part 3 video: short delivery cycles that each produce a usable increment.
- [Git Explained in 100 Seconds \(Fireship, YouTube\)](#) — Part 4 video: repository, staging, commit, branch.
- [Python Virtual Environments - Full Tutorial for Beginners \(Tech With Tim, YouTube\)](#) — Part 4 video: create and activate a venv on every operating system.
- [What are AI Agents? \(IBM Technology, YouTube\)](#) — Part 4 video: the general idea behind the JARVIS loop.

Before you click: paste the address into VirusTotal or Google Safe Browsing (Cyber.SoHo Notice, section 14 and Annex C).



One problem, three courses, one team.

See you in the workshop — Brick 1 starts now.

- By January, **JARVIS** logs and automates routine work, measures the time it saves with statistics, prices that time with microeconomics, and recommends what to automate first — and whether the free tier is enough.
- Six dates: November 13 · November 20 · December 4, 11, 18 · January 15 · January 22.
- 30 / 30 / 40 — twelve Bricks, the Mid-Term, the Product Defense.

© 2026 Cyber.SoHo — Educational Hub. All rights reserved. Educational content only. Use of this site and its content is subject to the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy. No affiliation with any vendor, platform or institution.

[Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy \(v2.0\) — open](#)

